# Операционные системы. Лабораторный практикум «Программирование в стандарте POSIX »

## *Задания и рейтинги*

| | Название | Тип.1 | Тип.2 | Тип 3 |
|---|---|---|---|---|
| 1 | Лаб.1. Процессы. Разделяемая память. Синхронизация | 8 | 10 | 13 |
| 2 | Лаб.2. Потоки. Синхронизация | 8 | 10 | 13 |
| 3 | Лаб.3. Процессы. Неименованные каналы | 8 | 12 | 15 |
| 4 | Лаб.4. Сетевые соединения. Сокеты | 8 | 12 | 15 |
| | Итог: лабы | **32** | **43** | **56** |
| 5 | РГР "Структуры данных и алгоритмы ОС" | 15 | | |
| 5 | РГР "Разработка простой файловой системы" | 20-25 | | |
| | Итог: РГР | **15-25** | | |
| 6 | Экзамен | 40 | | |

## *Настройка окружения для работы с POSIX под Windows*

Для настройки полноформатной POSIX желательно работать с ней под Linux (в Windows возможны некоторые ограничения или нестыковки). В Windows можно использовать интегрированный Linux, встроенный в виде виртуальной машины - WSL - Windows-Subsystem-for-Linux. Подробности настройки и запуска в презентации (WSL.pptx).

### Замечания и рекомендации

**Замечание.** Функция системного времени **clock (time.h)** в WSL не работает адекватно в потоках. Рекомендуется использовать функции из **sys/time.h** для POSIX-систем.

```
#include <sys/time.h>

timeval start_tv;

double diffMS(timeval &end){
    return end.tv_sec * 1. + end.tv_usec/1000000.
    - start_tv.tv_sec * 1. - start_tv.tv_usec/1000000.;
    }
```

```
int main(int argc, char *argv[]) {
    timeval end;
    gettimeofday(&start_tv, NULL);
    int size=100000000;
    printf("Размерность:%d Потоков: %d\n",size,MaxThreads);
    int *in=new int[size];
    int i;
    char ss[200];
    srand(time(NULL));
    for (i=0; i<size; i++) in[i]=rand()%(size/2);
    int nswap = sort(in,0,size-1);
    for (i=1; i<size && in[i-1]<=in[i]; i++);
    if (i!=size)
        printf("Не отсортированы элементы in[%d]=%d...%d\n",i-1, in[i-1],in[i]);
    gettimeofday(&end, NULL);
    sprintf(ss,"Обменов:%d Время выполнения:%5.3f мс", nswap,diffMS(end));
    perror(ss);
    puts("");
    exit(EXIT_SUCCESS);
    }
```

**Рекомендация.** Для запуска дерева процессов использовать заготовку **posix12.cpp**. Весь код родительского процесса и потомков в одном файле. При выполнении функции fork() потомки содержат клон адресного пространства (код и глобальные (статические) данные), локальные данные текущего потока (локальные переменные). Это можно использовать для неявной передачи параметров от родителя потомкам.

```
Родитель: pid=77 потомок=78
Порожденный процесс(1): родитель=77 потомок=78
Родитель: pid=77 потомок=78
Порожденный процесс(2): родитель=77 потомок=79
```

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main (void) {
    int childNum=1;
    int childId=fork();
    switch (childId) {
        case -1:
            perror ("FORK 1"); exit (2);
        case 0:
            // Порожденный процесс
            printf("Порожденный процесс(%d): родитель=%d потомок=%d\n",childNum, getppid(
            exit (0);
        default:
            printf("Родитель: pid=%d потомок=%d\n",getpid (),childId);
            //--------------------------- второй потомок -------------------------------
            childNum=2;
            switch (fork ()) {
                case -1:
                    perror ("FORK 2"); exit (2);
                case 0:
                    // Порожденный процесс
                    printf("Порожденный процесс(%d): родитель=%d потомок=%d\n",childNum,
                    exit (0);
                default:
                    printf("Родитель: pid=%d потомок=%d\n",getpid (),childId);
                    }
                }
            //-----------------------------------------------------------------------
        wait (NULL);
        return (0);
        }
```

## *Лаб.1. Процессы. Разделяемая память. Синхронизация*

Целью работы является программирование дерева порождающих процессов с использованием данных в общей разделяемой памяти. Процессы совместно реализуют рекурсивный алгоритм сортировки или независимые параллельны действия с частями структур данных. В случае обновления данных, используемых другими процессами, использовать семафоры (мьютексы).

Исходный код и описание алгоритмов сортировки взять с архива **cprog.rar** из папки **Programms (vk.com/cporg_cs -** подробности – ссылка на архив**).**

### Варианты заданий

**Тип 1.** Алгоритмы, обрабатывающие массив в общей памяти. Процесс-родитель создает процессы-потомки и передает каждому интервал массива. Потомок обрабатывает интервал и возвращает результат в общей памяти (отдельный раздел или продолжение раздела с данными). Используются семафоры для ожидания завершения всех потомков. Строится зависимость времени выполнения от количества пораженных процессов. Варианты заданий:

1. Поиск максимального и минимального значения на интервале. Процесс-потомок, получив все результаты, находит среди них максимальное и минимальное.
2. Вычисление среднего и дисперсии на интервале. Потомок вычисляет сумму и сумму квадратов, родитель вычисляет среднее и дисперсию всего массива данных.
3. Вычисление максимальной последовательности возрастающих значений (длина и индекс начала). Родитель вычисляет максимальное (с учетом того, что найденная последовательность может продолжаться на следующем интервале). При тестировании сгенерировать соответствующие данные.
4. Найти наименьшее общее кратное (НОК) всех элементов массива. Потомки вычисляют НОК каждый на своем интервале. Родитель объединяет полученные НОК, вычисляя для них НОК.
5. Удалить из массива все не простые числа (оставить простые). Потомки удаляют не простые числа из своих интервалов, уплотняя массив, и возвращают количество оставшихся. Родитель сливает полученные части в той же памяти.
6. Удалить из массива все отрицательные элементы. Потомки удаляют отрицательные числа из своих интервалов, уплотняя массив, и возвращают количество оставшихся. Родитель сливает полученные части в той же памяти.
7. Найти в массиве все пары соседних элементов, у которых последняя цифра первого элемента равна первой цифре второго. Потомки возвращают полученные значения, родитель суммирует их и добавляет пары на границе интервалов.
8. Найти в массиве все пары одинаковых, рядом стоящих элементов и удалить. Потомки удаляют пары элементов из своих интервалов, уплотняя массив, и возвращают количество оставшихся. Родитель сливает полученные части в той же памяти, учитывая наличие таких же пар на границах интервалов.

**Замечание.** Для тестирования программы сгенерировать данные, по которым можно проверить правильность путем сравнения результатов работы версии с единственным процессом и с группой процессов-потомков.

**Тип 2.** Нерекурсивные сортировки с однократным или циклическим созданием потомков. Рекурсивные сортировки с одним уровнем разделения.

9. 46-09. Сортировка Шелла с шагом по степеням 2. Родитель задает шаг сортировки и раздает потомкам индекс начала группы. Потомки сортируют каждый свою часть массива. По окончании работы всех потомков (семафоры) родитель уменьшает шаг и повторяет процесс, пока шаг не станет равным 1. Тогда он сортирует сам.

10. 46-10. Сортировка Шелла. Формула шагов h=3h+1. Аналогично предыдущему варианту.
11. 46-15. (62-04) Простое однократное слияние (на основе ДМУ). Родитель разбивает массив на N частей, после чего отдает каждый из них потомкам. После сортировки частей выполняет слияние.
12. 72-01. Сортировка массива рекурсивным разделением с одним уровнем разделения. Родитель выполняет разделение в массиве и отдает интервалы двум потомкам. Каждый выполняет сортировку рекурсивным разделением локально (без создания потомков, рекурсивным вызовом);
13. 72-02. "Быстрая" сортировка. Аналогично предыдущему варианту, другой способ разделения.
14. 91-04. Поразрядная сортировка разделением. Аналогично предыдущему варианту, Родитель разделяет массив по значению самого старшего ненулевого разряда и отдает интервалы и номер разряда двум потомкам. Каждый выполняет сортировку рекурсивным разделением локально (без создания потомков, рекурсивным вызовом);

**Тип 3.** Рекурсивное создание потомков до заданной глубины. Родитель выполняет разделение (или выделяет части массива для слияния) и передает из двум потомкам (интервал и уровень вложенности). Если уровень вложенности достигнут, то потомок выполняет сортировку локально, иначе создает двух потомков и выполняет аналогичные действия родителя.

15. 72-01. Сортировка массива рекурсивным разделением. Родитель передает потомкам интервал, уровень рекурсии.
16. 72-02. "Быстрая" сортировка. Аналогично предыдущему варианту, другой способ разделения.
17. 91-04. Поразрядная сортировка разделением. Аналогично предыдущему варианту, Родитель передает потомкам интервал, уровень рекурсии и номер разряда.
18. 72-04 Сортировка массива рекурсивным слиянием;
19. 72-03 Сортировка односвязного списка рекурсивным разделением. Родитель разделяет список на два, передает потомкам указатели на них.
20. 72-05 Сортировка односвязного списка рекурсивным слиянием
21. 46-15. (62-04) Однократное слияние в несколько уровней. Родитель раздает интервалы и сливает полученные упорядоченные массивы. Потомок при превышении уровня рекурсии сортирует список локально, иначе выполняет аналогичное разделение.
22. 72-04 Сортировка массива рекурсивным слиянием.

Задание реализовать в виде группы процессов, использующих разделяемую память. Корневой родительский процесс создает регион разделяемой памяти и структуру данных в ней (массив, список). Дальнейшие действия:

- корневой процесс получает массив и устанавливает глубину рекурсии, равную 0. Вызывает рекурсивную функцию сортировки для родителя, передавая весь массив;
- рекурсивная функция сортировки для родителя:
  - создает двух потомков, выполняет разделение массива и передает полученные части (диапазон) в адресном пространстве (глобальном/локальном) в момент порождения потомка;
  - после получения результатов от процессов-потомков выполняет соединение/слияние частей с обновлением результата в разделяемой памяти
- процесс-потомок присоединяется раздел разделяемой памяти и проверяет глубину рекурсии:

- если она достигла уровня ограничения – вызывает функцию-прототип для локальной сортировки;
- иначе вызывает рекурсивную функцию сортировки для родителя;

**Замечание.** При создании списковых структур (сортировка списков) необходимо учитывать, что динамическая память должна распределяться в регионе разделяемой памяти (*проблема*), либо необходимо создавать элементы списка в разделяемой памяти «вручную» с использованием адресной арифметики. Вторая проблема может возникнуть при несовпадении адресов разделяемой памяти в адресном пространстве различных процессов.

## Рекомендуемый порядок исполнения

Ввиду сложности отладки многопоточных программ и трассировки (отслеживания) состояния в Си, рекомендуется следующий порядок:

- «оживить» программу-образец с **cprog**;
- подобрать размерность входных данных, начиная с которых можно начитать сбор значимой статистики (десятки секунд);
- внедрить трассировку/тайминг – размерности сортируемых частей, момент входа/выхода, продолжительность;
- внести и протестировать примитивы запуска и завершения процессов (заготовка **posix15.cpp** –порождение двух процессов с присоединением к разделяемой памяти**);**
- реализовать целевой алгоритм;

```
Получить handle ----------------------1
Определить размер ----------------------2
Родитель: pid=268 Получить доступ--------------3
addr=7f04da6c4000
Заполнить память ----------------------4
3 13 29 28 34 49 47 22 49 9 1 45 13 26 37 23 48 29 44 27 28 17 22 38 40 0 2
 16 35 5 39 33 36 33 12 16 2 34 7 45 35 32 37 45 15 32 9 12 5 1 46 30 40 4
 28 19 27 17 15 43 0 26 7 7 28 5 37 18
Родитель: pid=268
Порожденный процесс(1): pid=269 предок=268 Получить доступ--------------5
addr=7f04da697000
3 13 29 28 34 49 47 22 49 9 1 45 13 26 37 23 48 29 44 27 28 17 22 38 40 0 2
 16 35 5 39 33 36 33 12 16 2 34 7 45 35 32 37 45 15 32 9 12 5 1 46 30 40 4
 28 19 27 17 15 43 0 26 7 7 28 5 37 18
Родитель: pid=268
Порожденный процесс(2): pid=270 предок=268 Получить доступ--------------5
addr=7f04da697000
3 13 29 28 34 49 47 22 49 9 1 45 13 26 37 23 48 29 44 27 28 17 22 38 40 0 2
 16 35 5 39 33 36 33 12 16 2 34 7 45 35 32 37 45 15 32 9 12 5 1 46 30 40 4
 28 19 27 17 15 43 0 26 7 7 28 5 37 18
```

- для ожидания завершения действий, выполняемых процессами-потомками, использовать семафоры. Заготовка **posix16.cpp** содержит код запуска двух процессов-потомков и использования пары семафоров для ожидания общего завершения;

```
Создать семафоры /posix16(1)_260 /posix16(2)_260
Значение семафоров (родитель)=0,0
Родитель: pid=260
Значение семафора потомок(1) до V-операции =0
Значение семафора потомок(1) после V-операции =1 t= 5.00
Значение семафора потомок(2) до V-операции =0
Значение семафора потомок(1) после V-операции =1 t=10.00
Значение семафора родитель после деблокировки =0,0 t=10.00
```

- внести трассировку/тайминг запуска/завершения процессов;
- зафиксировать моменты времени запуска/завершения процессов, моментов получения и отправки данных, построить временную диаграмму для 2-3 случаев.

Командные строки gcc-компиляции с ключами для потоков и динамической/разделяемой памяти и запуска

```
gcc -finput-charset=UTF8 -pthread posix15.cpp -o out.bin -lstdc++ -lrt
./out.bin
```

Пример порождения раздела разделяемой памяти, присоединения к разделу, запуска двух процессов потомков с присоединением к разделу приведен в заготовке **posix15.cpp.**

```
int main (void) {
    printf("Получить handle ----------------------1\n");
    shMemId  = shm_open(memName, O_CREAT|O_RDWR, 0777);
    if (shMemId==-1){
        perror("Ошибка создания памяти(shm_open)");
        return 1;
        }
    printf("Определить размер ----------------------2\n");
    ftruncate(shMemId, (shMemSize+1)*sizeof(int));
    printf("Родитель: pid=%d Получить доступ--------------3\n",getpid ());
    int *addr = (int*)mmap(0, (shMemSize+1)*sizeof(int), PROT_WRITE|PROT_READ, MAP_SHARED, shMemId, 0);
    printf("addr=%lx\n",(long unsigned int)addr);
    if ((char*)addr == (char*)-1 ) {
        perror("Ошибка связывания памяти (mmap)");
        return 1;
        }
    srand(time(NULL));
    printf("Заполнить память ----------------------4\n");
    for (int i=0; i<shMemSize; i++)
        addr[i]=rand()%(shMemSize/2);
    print(addr,shMemSize);
    int *addr2;
    int *addr3;
    switch (fork ()) {
        case -1:
            perror ("FORK 1"); exit (2);
        case 0:
            // Порожденный процесс
            printf("Порожденный процесс(%d): pid=%d предок=%d Получить доступ--------------5\n",childNum,get
            addr2 = (int*)mmap(0, (shMemSize+1)*sizeof(int), PROT_WRITE|PROT_READ, MAP_SHARED, shMemId, 0);
            printf("addr=%lx\n",(long unsigned int)addr2);
            print(addr2,shMemSize);
            exit (0);
        default:
            printf("Родитель: pid=%d\n",getpid ());
            childNum=2;
            switch (fork ()) {
                case -1:
                    perror ("FORK 2"); exit (2);
                case 0:
                    // Порожденный процесс
                    printf("Порожденный процесс(%d): pid=%d предок=%d Получить доступ--------------5\n",chil
                    addr3 = (int*)mmap(0, (shMemSize+1)*sizeof(int), PROT_WRITE|PROT_READ, MAP_SHARED, shMem
                    printf("addr=%lx\n",(long unsigned int)addr3);
                    print(addr3,shMemSize);
                    exit (0);
                default:
                    printf("Родитель: pid=%d\n",getpid ());
                    }
                }
            //-------------------------------------------------------------------------------------
        wait (NULL);
        return (0);
        }
```

## *Лаб.2. Потоки. Синхронизация*

Целью работы является программирование многопоточного приложения для рекурсивных алгоритмов сортировки или алгоритмов, допускающих независимые параллельны действия с частями структур данных.

Исходный код и описание алгоритмов сортировки взять с архива **cprog.rar** из папки **Programms (vk.com/cporg_cs -** подробности – ссылка на архив**).**

### Варианты заданий

Задание из лаб. работы 1 с реализацией в потоках.

Для оценки эффективности реализации в потоках выполнить определить размерность структуры данных, при которой получаются значимые результаты времени выполнения (порядка десятков секунд).

В алгоритмах рекурсивный вызов функции для полученных частей производить следуюоим образом:

- первый вызов выполняется в том же потоке, где выполняется текущий шаг рекурсии;
- второй вызов выполняется в зависимости от текущего количества активных потоков:
  - если он превышает установленное значение, то второй вызов выполняется в том же потоке, где выполняется текущий шаг рекурсии;
  - иначе создается новый поток (до начала первого рекурсивного вызова), ему передается интервал (список) и реализуются средства для ожидания завершения этого потока (семафор, сигнал). Текущий поток после первого рекурсивного вызова дожидается завершения созданного потока;
  - операции со счетчиком потоков (проверка/инкремент и декремент) синхронизируются счетчиком.

Для каждого варианта размерности сортировки выполнить ряд измерений с изменяющимся счетчиком потоков (0….N, 0 - в единственном текущем потоке). Построить графики зависимости, определить эффективное количество потоков.

В нерекурсивных алгоритмах создается план сортировки частей: за каждым потоком закрепляются несколько частей, потоки запускаются и происходит ожидание завершения их параллельной работы.

### Рекомендуемый порядок исполнения

Ввиду сложности отладки многопоточных программ и трассировки (отслеживания) состояния в Си, рекомендуется следующий порядок:

- «оживить» программу-образец;
- внедрить трассировку/тайминг – размерности сортируемых частей, момент входа/выхода, продолжительность;
- сгенерировать тестирующий код на последовательность размерностей в геометрической прогрессии (удвоение), определить размерности, начиная с которых можно начитать сбор статистики;
- внести и протестировать примитивы запуска и завершения потоков и их синхронизации (мьютекс счетчика) ;
- реализовать запуск потока для второй половины разделяемого диапазона с передачей через объект (struct) параметров сортируемой части (адрес, границы);

```
struct SortRequest{
    int *in,a,b;
    };
```

- внести трассировку/тайминг запуска/завершения потоков;

- протестировать программу с нулевым счетчиком разрешенных потоков (результаты совпадают с прототипом);
- получить статистику для различного количества потоков, построить графики зависимости, обосновать результат;
- в качестве результата рекурсивной функции возвратить параметр трудоемкости (количество сдвигов, сравнений, рекурсивных вызовов);
- зафиксировать моменты времени запуска/завершения потоков, построить временную диаграмму для 2-3 случаев.

Командные строки gcc-компиляции с ключами для потоков и динамической памяти и запуска

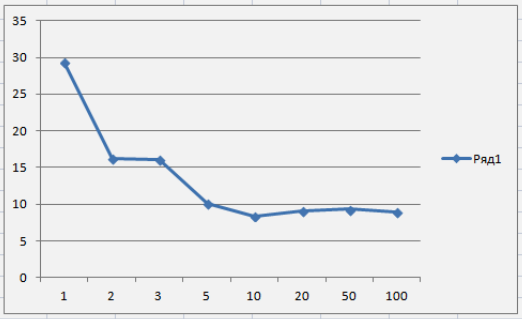
```
gcc -finput-charset=UTF8 -pthread lab3.cpp -o out.bin -lstdc++
./out.bin
```

Пример консольного вывода.

```
Размерность:100000000 Потоков: 5
Оригинал: Обменов:627762862 Время выполнения:28.648 мс
Потоковая функция стартует a=49998586 b=99999999 size=50001414 clock=1.637
Потоковая функция стартует a=24996643 b=49998585 size=25001943 clock=2.196
Потоковая функция стартует a=75004706 b=99999999 size=24995294 clock=2.198
Потоковая функция стартует a=62500152 b=75004705 size=12504554 clock=2.474
Потоковая функция завершена a=62500152 b=75004705 size=12504554 clock=6.555
Ожидание завершения к потоку a=49998586 b=75004705 clock=2.473
Поток вернул 69121356 clock=6.563
Ожидание завершения к потоку a=49998586 b=99999999 clock=2.198
Потоковая функция завершена a=75004706 b=99999999 size=24995294 clock=10.087
Поток вернул 144439872 clock=10.087
Потоковая функция завершена a=49998586 b=99999999 size=50001414 clock=10.087
Потоковая функция завершена a=24996643 b=49998585 size=25001943 clock=10.149
Ожидание завершения к потоку a=0 b=49998585 clock=2.195
Поток вернул 144450347 clock=10.165
Ожидание завершения к потоку a=0 b=99999999 clock=1.637
Поток вернул 301414271 clock=10.165
Обменов:627778223 Время выполнения:10.395 мс
```

Результаты измерений (время выполнения от количества потоков).

| Потоков | Время |
|---|---|
| 1 | 29,3 |
| 2 | 16,2 |
| 3 | 16,1 |
| 5 | 10,1 |
| 10 | 8,36 |
| 20 | 9,1 |
| 50 | 9,3 |
| 100 | 8,87 |




Пример использования мьютекса в заготовке **posix0-04**

```
int down=0;
pthread_mutex_t mutex;
void *thread(void *arg) {
    int num=*(int*)arg;
    printf("thread %d started\n", num);
    while(down==0){
        pthread_mutex_lock(&mutex);
        printf("%d a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a\n",num);
        printf("%d b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b\n",num);
        pthread_mutex_unlock(&mutex);
        }
    printf("thread %d finished\n", num);
    pthread_exit((void*)"!!!!!!!!!!!!!!!!!!!!!!!!");
    }

int main(int argc, char *argv[]) {
    int res;
    pthread_t a_thread;
    void *thread_result;
    int num=1;
    int num2=2;
    pthread_mutex_init(&mutex, NULL);
    res = pthread_create(&a_thread, NULL, thread, (void*)&num);
    if (res != 0) { perror("error 1"); exit(EXIT_FAILURE); }
    res = pthread_create(&a_thread, NULL, thread, (void*)&num2);
    if (res != 0) { perror("error 2"); exit(EXIT_FAILURE); }
    sleep(10);
    down=1;
    sleep(5);
    pthread_mutex_destroy(&mutex);
    exit(EXIT_SUCCESS);
    }
```

Для запуска потока с передачей параметров и возвращением результата использовать заготовку **posix03**

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

void *thread_function(void *arg);
char message[] = "Hello World";

int main(int argc, char *argv[]) {
    int res;
    pthread_t a_thread;
    void *thread_result;
    res = pthread_create(&a_thread, NULL, thread_function, (void*)message);
    if (res != 0) {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
        }
    printf("Waiting for thread to finish...\n");
    res = pthread_join(a_thread, &thread_result);
    if (res != 0) {
        perror("Thread join-failed");
        exit(EXIT_FAILURE);
        }
    printf("Thread-joined, it returned %s\n", (char *)thread_result);
    printf("Message is now %s\n", message);
    exit(EXIT_SUCCESS);
    }

void *thread_function(void *arg) {
    printf("thread_function is running. Argument was %s\n", (char *)arg);
    sleep(3);
    strcpy(message, "Bye!");
    pthread_exit((void*)"Thank you for the CPU time");
    }
```

### *Лаб.3. Процессы. Неименованные каналы*

Целью работы является программирование дерева порождаемых процессов с передачей данных между ними посредством каналов (pipe). Процессы совместно реализуют рекурсивный алгоритм сортировки или алгоритмов или же независимые параллельны действия с частями структур данных. Обмен необходимыми данными производится через программные каналы (pipe). Через передаваемые данные производится также синхронизация процессов.

Исходный код и описание алгоритмов сортировки взять с архива **cprog.rar** из папки **Programms (vk.com/cporg_cs -** подробности – ссылка на архив**).**

Варианты заданий

**Тип 1-1.** Линейный конвейер процессов. Длина конвейера варьируется. Процесс представляет собой фильтр с возможным промежуточным накоплением в FIFO. Производит фильтрацию текстового потока или потока двоичных целых от предыдущего процесса и передает отфильтрованный поток данных следующему процессу или сохраняет в файл по окончании цепочки. Проверить на файле литературного произведения (Например, «Война и мир» Л.Толстого) или на сгенерированной последовательности слов достаточного объема для оценки производительности:

1. Фильтр удаляет каждое n-ое слово из текстового потока. В потоке вначале передается количество счетчиков и сами счетчики. Процесс сохраняет первый передаваемый счетчик и передает следующему процессу оставшиеся в том же формате;
2. Фильтр переворачивает каждое n-ое слово из текстового потока. В потоке вначале передается количество счетчиков и сами счетчики. Процесс сохраняет первый передаваемый счетчик и передает следующему процессу оставшиеся в том же формате;
3. Фильтр удаляет заданное слово из текстового потока. В потоке вначале передается количество фильтруемых слов и сами слова. Процесс сохраняет первое передаваемое слово и передает следующему процессу оставшиеся в том же формате;
4. Фильтр удаляет самый внутренний фрагмент в фигурных скобках. В потоке вначале передается счетчик оставшихся процессов в конвейере. Процесс передает следующему этом счетчик за вычетом 1. При значении счетчика, равном 0, сохраняет в результат фильтрации в файл;
5. Фильтр удаляет младшую цифру любой целой константы в тексте. В потоке вначале передается счетчик оставшихся процессов в конвейере. Процесс передает следующему этом счетчик за вычетом 1. При значении счетчика, равном 0, сохраняет в результат фильтрации в файл;

**Тип 1-2.** Один родитель, несколько потомков. Процесс создает несколько потомков и передает им части массива, потомки выполняют требуемые действия и возвращают результат. Родитель принимает результаты и объединяет их в общий.

6. Поиск максимального и минимального значения на интервале. Процесс-потомок, получив все результаты, находит среди них максимальное и минимальное.
7. Вычисление среднего и дисперсии на интервале. Потомок вычисляет сумму и сумму квадратов, родитель вычисляет среднее и дисперсию всего массива данных.
8. Вычисление максимальной последовательности возрастающих значений (длина и индекс начала). Родитель вычисляет максимальное (с учетом того, что найденная последовательность может продолжаться на следующем интервале). При тестировании сгенерировать соответствующие данные.
9. Найти наименьшее общее кратное (НОК) всех элементов массива. Потомки вычисляют НОК каждый на своем подмассиве. Родитель объединяет полученные НОК, вычисляя для них НОК.

**Тип 2.** Нерекурсивные сортировки с однократным или циклическим созданием потомков. Рекурсивные сортировки с одним уровнем разделения. Передача массивов по каналам производится в формате – количество значений, сами значения.

9. 46-15. (62-04) Простое однократное слияние. Родитель разбивает массив на N частей, после чего отдает передает каждую часть потомку. После сортировки потомки возвращают отсортированные части, родитель выполняет слияние.
10. 72-01. Сортировка массива рекурсивным разделением с одним уровнем разделения. Родитель выполняет разделение в массиве и передает массивы двум потомкам. Каждый выполняет сортировку рекурсивным разделением локально (без создания потомков, рекурсивным вызовом) и возвращает отсортированные массивы обратно;
11. 72-02. "Быстрая" сортировка. Аналогично предыдущему варианту, другой способ разделения;
12. 91-04. Поразрядная сортировка разделением. Аналогично предыдущему варианту, Родитель разделяет массив по значению самого старшего ненулевого разряда и отдает интервалы и номер разряда двум потомкам. Каждый выполняет сортировку рекурсивным разделением локально (без создания потомков, рекурсивным вызовом);
10. Удалить из массива все не простые числа (оставить простые). Потомки получают части массива, удаляют не простые числа, и возвращают массив. Родитель соединяет полученные части;
11. Удалить из массива все отрицательные элементы. Аналогично предыдущему варианту;
12. Найти в массиве все пары соседних элементов, у которых последняя цифра первого элемента равна первой цифре второго. Аналогично предыдущему варианту. Родитель объединяет их и добавляет пары на границе интервалов;
13. Найти в массиве все пары одинаковых, рядом стоящих элементов и удалить. Аналогично предыдущему варианту. Родитель объединяет полученные части, учитывая наличие таких же пар на границах интервалов.

**Тип 3.** Рекурсивные алгоритмы, два потомка или более (по условию задачи). Рекурсивное создание потомков до заданной глубины.

13. 72-01. Сортировка массива рекурсивным разделением;
14. 72-02. "Быстрая" сортировка;
15. 91-04. Поразрядная сортировка разделением;
16. 46-15. (62-04) Однократное слияние в несколько уровней (на основе ДМУ). Процесс создает несколько потомков и передает им части массива. Они сортируют и возвращают отсортированные. Процесс выполняет многопутевое слияние. Потомки выполняют аналогичное действие на заданную глубину;
17. 72-04 Сортировка массива рекурсивным слиянием (два потомка);
18. Двоичное дерево. Процесс включает новое положительное значение в собственное дерево на заданную глубину, при превышении создает процесс – потомок и передает ему это значение. Отсортированные данные возвращаются отдельной командой – значением -1. Концевая вершина дерева содержит pid процессов-потомков.

Для выполнения следующего уровня сортировки (рекурсия) процесс порождает два канала и два дочерних процесса, передает им разделенные данные и получает обратно уже отсортированные. Родительский процесс производит соединение данных или слияние (в зависимости от вида сортировки).

Для синхронизации к началу/окончанию передачи данных через канал использовать избыточность кодирования данных и соответствующие форматы, например:

- счетчик количества передаваемых данных – данные;

- индикатор: конец последовательности (-1), данные (0, значение).

Для оценки эффективности реализации в потоках выполнить определить размерность структуры данных, при которой получаются значимые результаты времени выполнения (порядка десятков секунд).

Ввести ограничение на глубину рекурсии, при которой происходит разделение на процессы. При ее достижении сортировка выполняется как обычная рекурсивная в рамках одного процесса.

## Рекомендуемый порядок исполнения

Ввиду сложности отладки многопоточных программ и трассировки (отслеживания) состояния в Си, рекомендуется следующий порядок:

- «оживить» программу-образец;
- внедрить трассировку/тайминг – размерности сортируемых частей, момент входа/выхода, продолжительность;
- сгенерировать тестирующий код на последовательность размерностей в геометрической прогрессии (удвоение), определить размерности, начиная с которых можно начитать сбор статистики;
- внести и протестировать примитивы запуска и завершения процессов и передачи данных через каналы и синхронизации через сигналы;
- внести трассировку/тайминг запуска/завершения процессов;
- протестировать программу с нулевым уровнем порождения процессов (результаты совпадают с прототипом);
- получить статистику для различного числа уровней рекурсии с порождением процессов, построить графики зависимости, обосновать результат;
- зафиксировать моменты времени запуска/завершения процессов, построить временную диаграмму для 2-3 случаев.

Пример порождения процесса в заготовке **posix0-07**

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>
void atefunc (void) {
    printf ("Ситуация перед завершением родительского процесса\n");
        system ("ps");
    }
int main (void) {
    int pid;
    int stat;
    setbuf (stdout, NULL);
    /* Зарегистрируем обработчик завершения процесса */
    if (atexit (atefunc) != 0) { perror ("ATEXIT"); exit (1); }
    /* Создадим новый процесс */
    if ((pid = fork ()) < 0) { perror ("FORK"); exit (2);}
        else if (pid == 0) {
            /* Порожденный процесс - Выполним служебную программу ps */
            printf ("Ситуация с точки зрения порожденного процесса\n");
            execl ("/bin/ps", "ps", (char *) 0);
            perror ("EXEC");
            exit (3); /* execl() завершился неудачей */
            } else {
        /* Родительский процесс */
        sleep (1);
        /* Вероятно, порожденный процесс уже завершился */
        /* Посмотрим на текущую ситуацию */
        printf ("Ситуация перед вызовом waitpid() в родительском процессе\n");
        system ("ps");
        waitpid (pid, &stat, 0);
        printf ("Статус завершения порожденного процесса с идентификатором %d:
                }
    return 0;
    }
```

Пример порождения процесса с созданием канала и обменом данными в заготовке **posix0-09**

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main (void) {
    int fd [2];
    int A[]={1,2,3,4,5,6,7,8,9,10};
    int B[10];
    /* —оздадим безым¤нный канал */
    if (pipe (fd) < 0) { perror ("PIPE"); exit (1); }
    switch (fork ()) {
    case -1:  perror ("FORK"); exit (2);
    case 0:
            close (fd [1]);
            if (read (fd [0], B, sizeof(int)*10) == sizeof(int)*10) {
                for(int i=0;i<10;i++)
                    printf("%d ",B[i]);
                }
            printf("\n");
            exit (0);
            }
    close (fd [0]);
    if (write (fd [1], A, sizeof(int)*10) != sizeof(int)*10) {
        perror ("WRITE TO PIPE-1");
        }
    close (fd [1]);
    wait (NULL);
    return (0);
    }
```

**Замечание.** Ввиду того, что неименованный канал можно использовать только для односторонней передачи (закрывать по одному дескриптору из каждой пары родителем и потомком), необходимо создать 4 канала.

```
int size=1000000;
int fd1req[2];          // Канал запроса: данные родитель/потомок 1
int fd1ans[2];          // Канал ответа: данные потомок/родитель 1
int fd2req[2];          // Канал запроса: данные родитель/потомок 2
int fd2ans[2];          // Канал ответа: данные потомок/родитель 2
```

**Замечание.** Размер буфера, выделяемого неименованному каналу, ограничен, причем при записи в канал данных, превышающих его размер, происходит блокирование процесса. При чтении процесс получает только данные, находящиеся в буфере, поэтому в программе необходимо чтение данных «по частям» и их размещение в собственном буфере приема.

```
    }
//-------------------------- Чтение по полученным частям, ограниений нет пока
int readMore(int fd, int *in, int size){
    char *a = (char*)in;            // Чтение побайтное
    int rest=size*sizeof(int);
    int idx=0;
    int step=0;
    while(1){
        step++;
        int vv = read(fd,a+idx,rest);
        idx += vv;
        rest -= vv;
        printf(".......pid=%d fd=%d step=%d size=%d idx=%d rest=%d\n",getpid()
        if (rest==0){
            return step;
            }
        if (rest<0){
            return -step;
            }
        }
    }
```

```
Порожденный процесс (шаг 1): pid=244 fd=13,16
.......pid=243 fd=8 step=1 size=64720 idx=64720 rest=933888
.......pid=243 fd=8 step=2 size=65536 idx=130256 rest=868352
.......pid=243 fd=8 step=3 size=65536 idx=195792 rest=802816
.......pid=243 fd=8 step=4 size=65536 idx=261328 rest=737280
.......pid=243 fd=8 step=5 size=65536 idx=326864 rest=671744
.......pid=243 fd=8 step=6 size=65536 idx=392400 rest=606208
.......pid=243 fd=8 step=7 size=65536 idx=457936 rest=540672
.......pid=243 fd=8 step=8 size=65536 idx=523472 rest=475136
.......pid=243 fd=8 step=9 size=65536 idx=589008 rest=409600
.......pid=243 fd=8 step=10 size=65536 idx=654544 rest=344064
.......pid=243 fd=8 step=11 size=65536 idx=720080 rest=278528
.......pid=243 fd=8 step=12 size=65536 idx=785616 rest=212992
.......pid=243 fd=8 step=13 size=65536 idx=851152 rest=147456
.......pid=243 fd=8 step=14 size=65536 idx=916688 rest=81920
.......pid=243 fd=8 step=15 size=65536 idx=982224 rest=16384
Родитель* (шаг 3): pid=240 fd=9,11 size=249652
Родитель* (шаг 4): pid=240 fd=9,11 size=249652
.......pid=243 fd=8 step=16 size=16384 idx=998608 rest=0
```

Пример консольного вывода.

```
Размерность:100000000 Вложенность процессов: 2 sizeof(int)=4
Оригинал: обменов:627773583 Время выполнения:28.671 мс
Родитель: pid=430
Порожденный процесс (шаг 1): pid=431 fd=3,6 t= 1.82
Родитель (шаг 1): pid=430 fd=4,5 sz=50000289 t= 1.82
Порожденный процесс (шаг 1): pid=432 fd=7,10 t= 1.82
Родитель (шаг 2): pid=430 fd=8,9 sz=49999711 t= 2.32
Родитель: pid=431
Порожденный процесс (шаг 1): pid=433 fd=4,8 t= 2.70
Родитель (шаг 1): pid=431 fd=5,7 sz=25000600 t= 2.70
Порожденный процесс (шаг 1): pid=434 fd=9,12 t= 2.70
Родитель (шаг 2): pid=431 fd=10,11 sz=24999689 t= 2.95
Родитель: pid=432
Порожденный процесс (шаг 1): pid=435 fd=8,12 t= 2.95
Родитель (шаг 1): pid=432 fd=9,11 sz=25006394 t= 2.95
Порожденный процесс (шаг 1): pid=436 fd=13,16 t= 2.95
Родитель (шаг 2): pid=432 fd=14,15 sz=24993317 t= 3.21
Порожденный процесс (шаг 6): потомок-1 pid=433 size=25000600  t= 9.71
Порожденный процесс (шаг 6): потомок-2 pid=434 size=24999689  t=10.07
Родитель (шаг 3): pid=431 fd=10,11 sz=24999689 t=10.07
Порожденный процесс (шаг 6): потомок-2 pid=435 size=25006394  t=10.25
Порожденный процесс (шаг 6): потомок-2 pid=436 size=24993317  t=10.39
Родитель (шаг 3): pid=432 fd=14,15 sz=24993317 t=10.39
Порожденный процесс (шаг 6): потомок-2 pid=431 size=50000289  t=10.45
Порожденный процесс (шаг 6): потомок-2 pid=432 size=49999711  t=10.74
Родитель (шаг 3): pid=430 fd=8,9 sz=49999711 t=10.74
Обменов:0 Время выполнения:11.129 мс
```

### *Лаб.4. Сетевые соединения. Сокеты*

Задание из лаб. работы 1 с реализацией на сокетах. Родительский процесс создает ServerSocket для приема локальных соединений ip=127.0.0.1 (localhost) с номером порта, совпадающим с его pid. Порождает два процесса, которые получают pid родителя и по одноименному порту связываются с ним. Далее процесс обмена данными и сортировки идентичен:

- корневой процесс получает массив и устанавливает глубину рекурсии, равную 0. Вызывает рекурсивную функцию сортировки для родителя, передавая весь массив;
- рекурсивная функция сортировки для родителя:
  - создает двух потомков, устанавливает с ними соединение, выполняет разделение массива и передает полученные части через соединение в формате: глубина рекурсии+1 / размерность массива / массив;
  - после получения результатов от процессов-потомков выполняет соединение/слияние частей;
  - возвращает отсортированные данные;
- процесс-потомок при получении массива проверяет глубину рекурсии:
  - если она достигла уровня ограничения – вызывает функцию-прототип для локальной сортировки;
  - иначе вызывает рекурсивную функцию сортировки для родителя;
  - в обоих случаях результат отправляет родителю в соединение в формате: счетчик обменов / размерность массива / массив;

Рекомендуемый порядок исполнения

Ввиду сложности отладки многопоточных программ и трассировки (отслеживания) состояния в Си, рекомендуется следующий порядок:

- «оживить» программу-образец с **cprog**;
- подобрать размерность входных данных, начиная с которых можно начитать сбор значимой статистики (десятки секунд);
- внедрить трассировку/тайминг – размерности сортируемых частей, момент входа/выхода, продолжительность;
- внести и протестировать примитивы запуска и завершения процессов (заготовка **posix12.cpp –**порождение двух процессов**);**
- протестировать заготовку **posix14.cpp** с функциями серверного сокета (прием соединения), сокета родителя (ping: отправка и обратный прием массива ) и потомка (прием и отправка массива). **Замечание:** для идентификации порта родителя он устанавливается равным ServerSocketBase + <pid родителя>;
- объединить **posix12.cpp** и **posix14.cpp** реализацией ping от родителя к двум потомкам;
- реализовать целевой алгоритм;
- внести трассировку/тайминг запуска/завершения процессов;
- зафиксировать моменты времени запуска/завершения процессов, моментов получения и отправки данных, построить временную диаграмму для 2-3 случаев.

Пример работы с сокетами в **posix14.cpp.**

Пример консольного вывода.

```
Размерность:100000000 Вложенность процессов: 3 sizeof(int)=4
Оригинал: Обменов:627758912 Время выполнения:28.746 мс
Родитель pid=6168
Потомок=6169, родитель pid=6168
Потомок=6170, родитель pid=6168
+++Потомок=6169, родитель pid=6168, потомок получил level=2 массива[50004325] t= 2.67
Родитель pid=6169
Потомок=6171, родитель pid=6169
Потомок=6172, родитель pid=6169
+++Потомок=6170, родитель pid=6168, потомок получил level=2 массива[49995675] t= 2.79
Родитель pid=6170
Потомок=6173, родитель pid=6170
Потомок=6174, родитель pid=6170
+++Потомок=6171, родитель pid=6169, потомок получил level=3 массива[25002176] t= 3.75
+++Потомок=6172, родитель pid=6169, потомок получил level=3 массива[25002149] t= 3.82
+++Потомок=6173, родитель pid=6170, потомок получил level=3 массива[25004733] t= 3.89
+++Потомок=6174, родитель pid=6170, потомок получил level=3 массива[24990942] t= 3.95
+++Потомок=6171, родитель pid=6169, после сортировки t=10.69
Родитель pid=6169, родитель получил массив[25002176] t=10.73
+++Потомок=6172, родитель pid=6169, после сортировки t=10.79
Родитель pid=6169, родитель получил массив[25002149] t=10.82
+++Потомок=6173, родитель pid=6170, после сортировки t=10.85
+++Потомок=6174, родитель pid=6170, после сортировки t=10.90
Родитель pid=6170, родитель получил массив[25004733] t=10.91
Родитель pid=6170, родитель получил массив[24990942] t=10.96
+++Потомок=6169, родитель pid=6168, после сортировки t=10.96
Родитель pid=6168, родитель получил массив[50004325] t=11.04
+++Потомок=6170, родитель pid=6168, после сортировки t=11.08
Родитель pid=6168, родитель получил массив[49995675] t=11.14
Обменов:577755271 Время выполнения:11.590 мс
```

**Замечание.** При проведении измерений нужно учитывать, что в каждом процессе используется только один поток, поэтому раздача и прием разделенный данных (если он выполняется накопленными массивами) производится последовательно для первогои второго потомка.

**Замечание.** Для исключения зависания при передаче накопленных данных в TCP-соединение рекомендуется отключить ожидание накопления данных в FIFO (алгоритм Найгля), а также установить параметр блокирующего чтения на приеме.

```
    }
int on=1;
setsockopt(client1, IPPROTO_TCP, TCP_NODELAY, &on, sizeof( on ) );
setsockopt(client2, IPPROTO_TCP, TCP_NODELAY, &on, sizeof( on ) );
level++;
```

```
    }
bytes_read = recv(socket, &size2, sizeof(int), MSG_WAITALL);
if (bytes_read!=sizeof(int)){
    printf("Родитель pid=%d, родитель принял для size %d байт\n"
    close(socket);
    return 0;
    }
```